



Explainable Enhanced Recurrent Neural Network for lie detection using voice stress analysis

Fatma M. Talaat^{1,2} 

Received: 31 March 2022 / Revised: 21 July 2023 / Accepted: 31 August 2023 /
Published online: 20 September 2023
© The Author(s) 2023

Abstract

Lie detection is a crucial aspect of human interactions that affects everyone in their daily lives. Individuals often rely on various cues, such as verbal and nonverbal communication, particularly facial expressions, to determine if someone is truthful. While automated lie detection systems can assist in identifying these cues, current approaches are limited due to a lack of suitable datasets for testing their performance in real-world scenarios. Despite ongoing research efforts to develop effective and reliable lie detection methods, this remains a work in progress. The polygraph, voice stress analysis, and pupil dilation analysis are some of the methods currently used for this task. In this study, we propose a new detection algorithm based on an Enhanced Recurrent Neural Network (ERNN) with Explainable AI capabilities. The ERNN, based on long short-term memory (LSTM) architecture, was optimized using fuzzy logic to determine the hyperparameters. The LSTM model was then created and trained using a dataset of audio recordings from interviews with a randomly selected group. The proposed ERNN achieved an accuracy of 97.3%, which is statistically significant for the problem of voice stress analysis. These results suggest that it is possible to detect patterns in the voices of individuals experiencing stress in an explainable manner.

Keywords Lie detection · Voice stress analysis · Deep recurrent neural network · Affective computing

1 Introduction

Humans are said to have a difficult time detecting deception. To explain why it is so difficult for us, Ekman [1] cites five reasons: (i) Throughout most of human history, there have been smaller civilizations where liars would have had a higher probability of getting found and suffering severe repercussions than they do now. (ii) Because their parents' desire to keep some things hidden from them, children are not taught how to

✉ Fatma M. Talaat
fatma.nada@ai.kfs.edu.eg

¹ Faculty of Artificial Intelligence, Kafrelsheikh University, Kafrelsheikh 33516, Egypt

² Faculty of Computer Science & Engineering, New Mansoura University, Gamasa 35712, Egypt

identify lying. (iii) People would rather stay ignorant of the truth. (iv) People would rather believe what they are taught. (v) People would rather believe what they are taught.

However, it has been suggested that with enough feedback (e.g., that the person is lying 50% of the time) and a focus on micro-expressions, someone can learn to identify lies in another person [1, 2]. Based on the preceding, facial analysis has been shown to be capable of detecting dishonest behavior through the use of macro- and, in particular, micro-expressions [3–5]. On the other hand, micro-expressions are difficult to capture at standard frame rates, and although people can learn to notice them in order to detect lies, liars may use the same training to learn how to hide them.

As a result, there's been an interest in detecting misleading face patterns that aren't visible to the naked eye, such as the heat signature of the periorbital [6] or perinasal area [7] in thermal images, which can't be seen with human vision. One of the most important aspects of properly addressing lie-detection research is the availability of adequate datasets, which is a key component of open innovation in speeding up current research as opposed to closed or private datasets, which characterize the opposite counterpart (closed innovation) [8].

Regardless of current advancements, acquiring training and, notably, assessment materials for lie detection remains a difficult issue, especially given the need to obtain ground truth, i.e., knowing if a person is lying or not [9]. The fundamental issue derives from the fact that such knowledge is useless if the scenario is naively replicated (e.g., it is not sufficient to instruct a person to simply tell a lie). As a result, scientists have attempted to develop artificial setups that can successfully mimic circumstances in which two elements collide: (i) there is the possibility of sincere deception. (ii) The majority of previous attempts have focused on interview scenarios in which participants are instructed to lie [6, 7, 10], despite the difficulty of simulating a realistic setting for true deception.

Alternatively, some academics have collaborated with police departments, resulting in a scenario that is 100 percent realistic in many circumstances because it is based on interviews with criminal suspects. However, the ground truth is an issue in this situation: it is impossible to rely on legal decision-making [11], and the veracity of confessions has been questioned [12]. In 1921, John Larson invented the first polygraph. It's a piece of equipment that can tell if someone is lying. It detects physiological disturbances in the target person's body.

Telling a lie, according to a polygraph, causes stress, which can be detected [13]. The signals recorded from the target are written on a paper ribbon in the standard polygraph apparatus. Respiratory frequency, heart rate, blood pressure, and sweating can all be represented by the signals. Polygraphs that read the movement of the arms and legs are available. The findings on the ribbon were reviewed by an interpreter to determine whether the target's answers were reliable [14]. In most cases, a polygraph is not used to determine whether or not someone is lying in a critical situation because there is no clear assurance that it will not fail.

To address these issues, some software versions of polygraphs have been proposed. Voice stress analysis is used in some versions [15]. Through voice stress analysis, a subject's stress or threat is detected. His muscles are ready to respond when his body reacts. Because of the tension in the respiratory system and tissues, these preparations also affect the voice. As a result, stress can be detected via the voice [16]. The goal of VSA (Voice Stress Analysis) software is to measure the disturbances in a subject's voice pattern. Physical stress that is induced when lying can cause them. The software analyses the vocal disturbances and produces a result [15].

Physical stress that is induced when lying can cause them. The software analyses the vocal disturbances and produces a result. Long Short-Term Memory (LSTM) neural networks were designed, constructed, and trained to accomplish this. The results reveal some surprising levels of precision. Given the circumstances, the study challenge can be reduced to a single question: "Would it be possible to identify a lie in a person's speech by using a neural network to analyze the stress in his voice during his speech?" If that's the case, how substantial can the outcomes be? ".

Explainable Artificial Intelligence (XAI) has become increasingly important in recent years due to the growing need for transparency and accountability in AI-based decision-making systems. The traditional "black box" approach of many machine learning algorithms can make it difficult for users to understand how the system arrived at a particular decision, which can lead to distrust and hesitation in adopting such systems [17].

By incorporating XAI into the design of AI-based systems, researchers and developers can provide a more transparent and interpretable model for decision-making. This not only helps users to understand the logic behind the system's output but also enables them to detect and correct any biases or errors in the decision-making process. Furthermore, XAI can facilitate the communication between users and AI systems, leading to improved trust, better decision-making, and more informed judgments [18].

In the context of lie detection, the use of XAI can provide valuable insights into how the system is able to identify patterns in the voices of individuals experiencing stress. By understanding the reasoning behind the system's decision-making process, researchers and users can gain a better understanding of the cues and features that are most relevant to identifying deception. This can help to improve the accuracy and reliability of the system, as well as provide greater confidence in its ability to detect lies.

Overall, the incorporation of XAI into AI-based systems is critical to ensuring the trustworthiness and effectiveness of these systems. By enabling users to understand and interpret the decisions made by AI systems, XAI can promote greater transparency, accountability, and reliability in the development and deployment of AI-based systems.

This paper introduces a novel lie detection algorithm that utilizes an Explainable Enhanced Recurrent Neural Network (ERNN). The ERNN is based on the Long Short-Term Memory (LSTM) architecture, and its hyperparameters are optimized using Fuzzy Logic. The training of the LSTM model is done using a dataset of audio recordings obtained from interviews with a randomly selected group, with corresponding output values. The proposed ERNN achieves a high accuracy of 97.3% in detecting lies using voice stress analysis. The use of Explainable Artificial Intelligence (XAI) in the proposed algorithm allows for the interpretation and explanation of the results, making it easier for users to understand and trust the system. This contribution is significant as it addresses the issue of interpretability and transparency, which is essential for the practical application of AI systems in sensitive domains such as lie detection.

The main contributions of this paper are:

- Proposing an Enhanced Recurrent Neural Network (ERNN) with Explainable AI capabilities for lie detection based on voice stress analysis.
- Optimizing the ERNN using fuzzy logic to determine the hyperparameters, which helps in achieving better accuracy.
- Creating and training the LSTM model using a dataset of audio recordings from interviews with a randomly selected group.
- Achieving an accuracy of 97.3%, which is statistically significant for the problem of voice stress analysis.

- Providing an explainable AI approach that allows for better interpretation and understanding of the decision-making process in the algorithm.

The remaining work is organized as follows: Section 2 discusses about related works. In Section 3, the proposed method is presented. Experimental evaluation is provided in Section 4. And in Section 5, we conclude this work.

2 Related work

Liu's work serves a similar function as the current work [16]. Its goal is to identify voice stress levels using pitch and jitter while using Matlab to do Bayesian hypothesis testing. The best result achieved for this study was an accuracy rate of 87 percent. However, this result could only be achieved with a dependent speaker, meaning that the model only behaved this way when analyzing the voice of a single individual and employing the Pitch voice feature. Using the Pitch characteristic, the accuracy value for the multi-speaker test (independent speaker) reduces to 70%.

The goal of [19]'s research is to identify lies by examining and analyzing the pupil dilation of the human eye using a neural network and the backpropagation method. The 60 preprocessed data were fed into a neural network. We examined networks with different numbers of hidden layers: 2, 7, 10, 20, and 50. The 10 layer network produced the best results. During training, the network properly classified all sample images. With these findings, it was discovered that, while not tested with new photos, it is feasible to reliably classify photographs of pupils based on their dilation.

Chow and Louie [20] want to use speech processing and natural language to detect a person's deception using audio obtained from the subject. This was accomplished with the help of an LSTM neural network. A single-level, one-way drop-out Long-Short Term Memory model was used as the final recurrent neural network model (LSTM). The accuracy of this method of implementation was roughly 61 percent to 63 percent. The MFCC was chosen as the voice feature for analysis. In the work of Dede et al. [21], the objective is to perform isolated speech recognition, employing the use of neural networks, where the recognition of digits 0 to 9 uttered by a speaker is performed.

The MFCC was chosen as the voice feature for analysis. The voice recognition system built in this research detected the digits quite well in the tests, with an Elman network hit rate of 99.35 percent, a PNN network hit frequency of 100 percent, and an MLP network hit rate of 98.75 percent. The findings of this study were quite encouraging, indicating that Artificial Neural Networks are a suitable and effective method for voice recognition.

There are several other studies that are relevant to the topic of automated lie detection using voice stress analysis.

The presence of psychological stress is considered a pathological aspect of this condition, and its cause is believed to be related to workload. In order to find objective and measurable indicators of stress, researchers have investigated changes in voice characteristics caused by workload. Several voice features such as loudness, fundamental frequency, jitter, zero-crossing rate, speech rate, and high-energy frequency ratio are thought to be affected by stress. To investigate the impact of workload on speech production, an experiment was designed, which involved 108 Dutch native speakers taking a stress test (Stroop test). This paper reports on the experiment and the analysis of the test results [22].

Another study by Ben-Shakhar and Elaad [23] focused on voice stress analysis using the polygraph, and achieved a sensitivity of 74.5% and specificity of 73.5%. They concluded that while the polygraph has limitations, it is still a useful tool for detecting deception.

Lie detection is a field that has been evolving over time with the use of linguistic analysis, recognition of facial and body movements, training observation, and voice stress analysis. With the advent of cognitive science and neuroscience, EEG analysis has provided a better understanding of brain function. Machine learning techniques such as SVM, k-Means, ANN, and Linear Classifier have been employed to analyze EEG signals. The Fast Fourier Transform (FFT) technique is also used to eliminate noise in the signals obtained from EEG. This literature review focuses on the connection between these three fields: EEG, machine learning, and lie detection. Authors in paper [24] cover topics such as measuring EEG signals, analyzing EEG signals, extracting features, using EEG for lie detection, classification algorithms, approaches, and methods used for EEG signal analysis in lie detection, as well as reviews and conclusions.

A lot of researchs have been done in this area. In [25], Han et al. propose an innovative ERNN model for lie detection based on voice stress analysis. The ERNN architecture, built upon the Long Short-Term Memory (LSTM) framework, is optimized using fuzzy logic to determine the hyperparameters. The model achieves promising accuracy in detecting deceptive speech patterns using voice stress analysis and incorporates Explainable Artificial Intelligence (XAI) techniques to provide interpretable insights into its decision-making process.

Kim et al. [26] explore the combination of convolutional neural networks (CNN) and LSTM for lie detection using voice stress analysis. The proposed model analyzes voice features and speech patterns to detect deceptive behavior. The study highlights the importance of using multimodal deep learning techniques for more accurate lie detection and provides insights into the effectiveness of CNN-LSTM fusion.

Xie et al. [27] introduce a multimodal deep learning approach for lie detection, combining information from multiple sources, including voice stress analysis, facial expressions, and textual content. The authors leverage advanced neural network architectures to learn complex patterns and correlations between modalities, enhancing the accuracy of lie detection while promoting interpretability.

Almatarneh et al. [28] propose an ensemble approach using multiple LSTM networks to improve the robustness and accuracy of lie detection based on voice stress analysis. By combining the predictions of individual LSTM models, the ensemble method achieves higher performance, addressing the issue of overfitting and enhancing the generalization capability of the model.

Goyal et al. [29] focus on the incorporation of attention mechanisms into recurrent neural networks for lie detection using voice stress analysis. The attention-based approach allows the model to dynamically focus on informative features and time steps during the analysis, leading to improved accuracy and explainability in detecting deceptive speech patterns.

Sun et al. [30] extend the lie detection task to a multimodal setting, combining audio and video signals. The proposed model employs both convolutional and recurrent neural networks to analyze and fuse information from multiple modalities, demonstrating the potential of multimodal analysis for more comprehensive lie detection.

Sharma et al. [31] introduce the use of bi-directional LSTM neural networks for voice stress analysis in deception detection. The bi-directional approach allows the model to capture information from both past and future time steps, enhancing its ability to identify patterns indicative of deception in speech.

Kang et al. [32] focus on improving the explainability of lie detection models using attention mechanisms in recurrent neural networks. The attention-based approach allows the model to highlight critical features and decision-making steps, providing transparent and interpretable results in voice stress analysis.

Yang et al. [33] propose the use of bidirectional gated recurrent units (GRU) for lie detection using voice stress analysis. The bidirectional architecture enhances the model's ability to capture contextual information and long-term dependencies in speech data, improving the accuracy of deceptive speech pattern detection.

Saxena et al. [34] present a novel lie detection approach using attention-based LSTM neural networks on voice stress analysis. The model leverages attention mechanisms to focus on crucial time steps in speech signals, enabling better understanding and interpretation of the model's decision-making process.

Table 1 illustrates the most frequently employed models proposed for lie detection based on voice stress, such as Linear Regression Model, Decision Tree Model, Random Forest Model, and Neural Networks (NN).

There are some challenges in the previous models such as: (i) Difficulty in obtaining a large and diverse dataset for training the model. (ii) Over fitting can occur due to the complexity of the model and the limited dataset. (iii) Lack of interpretability of the results and difficulty in understanding the reasoning behind the model's decision. (iv) Limited ability to handle temporal dependencies and capture long-term dependencies in the input data.

The research gap can be summarized as follows:

- Limited research has been conducted to evaluate the performance of ANN, RNN, and LSTM models for lie detection based on voice stress in real-world scenarios.
- The existing studies do not provide a comprehensive comparison of the performance of different machine learning algorithms for this task.
- The studies do not address the issue of feature selection and extraction, which can have a significant impact on the performance of the model.

This research aims to address this gap by developing proposed algorithm, called Enhanced Recurrent Neural Network (ERNN), incorporates fuzzy logic to optimize the hyperparameters of the Long Short-Term Memory (LSTM) architecture. In doing so, we hope to achieve the following:

- This approach improves the model's ability to capture long-term dependencies and handle noisy and incomplete input data.
- Additionally, we conducted experiments on a diverse and real-world dataset to evaluate the performance of the model and compared it with the state-of-the-art algorithms.
- Furthermore, we explored different voice features and used feature selection techniques to identify the most relevant features for lie detection. These efforts resulted in an accurate and reliable model for lie detection based on voice stress.

3 Explainable Enhanced Recurrent Neural Network (ERNN)

This paper proposes a new detection algorithm based on enhanced recurrent neural network. An Enhanced Recurrent Neural Network (ERNN) of LSTM architecture was implemented (Fig. 1). The proposed algorithm can be grouped into seven main modules:

Table 1 Comparison results of the most common used models

Model	Algorithm Description	Pros	Cons
Artificial Neural Networks (ANN)	Artificial Neural Networks (ANN) used for lie detection based on voice stress involve training a neural network to classify audio recordings as truthful or deceptive based on voice features such as pitch, jitter, and spectral analysis. The neural network learns to detect patterns in these features and make predictions based on them	• ANNs can handle large amounts of data and can learn complex relationships between features • They are flexible and can be adapted to different datasets and contexts • They can provide real-time detection and can be integrated with other technologies	• ANNs require large amounts of training data to be effective • They can be sensitive to noise and variability in the input data • The results of ANNs can be difficult to interpret and may not provide insights into the underlying causes of deception
	Recurrent Neural Network (RNN) for lie detection based on voice stress is trained using a dataset of audio recordings from interviews with a selected group of individuals. The RNN takes in the audio data as input and outputs a binary value indicating whether the speaker is lying or not	• RNNs are well-suited for sequential data, which is appropriate for analyzing speech signals • RNNs can model the temporal dependencies in the input data, which is important for capturing patterns of deception over time	• RNNs can suffer from the vanishing gradient problem, which can make it difficult for the network to learn long-term dependencies • RNNs can be computationally expensive to train and require large amounts of training data to achieve high performance
LSTM Neural Network	The LSTM (Long Short-Term Memory) neural network algorithm used for lie detection based on voice stress is a type of recurrent neural network that is capable of handling long-term dependencies in sequential data. It uses a single-level, one-way dropout LSTM model as the final recurrent neural network model. The input to the algorithm is a set of audio recordings from interviews with subjects, and the algorithm analyzes the speech patterns to detect signs of deception	• LSTM networks are well suited to analyze sequential data, making them effective for lie detection based on voice stress • The one-way dropout approach used in this algorithm helps prevent over fitting and improves the generalization ability of the model • The algorithm achieved an accuracy rate of roughly 61 percent to 63 percent in detecting deception in speech	• The accuracy rate achieved by the algorithm is relatively low compared to some other techniques used for lie detection • The algorithm may be susceptible to errors due to variations in individual speaking styles, making it less reliable in real-world scenarios

(i) **Data Collection:** Collect audio recordings from interviews with a randomly selected group of individuals. (ii) **Data Preprocessing:** Prepare the audio recordings by converting them into a suitable format for training the ERNN model. (iii) **Feature Extraction:** Extract relevant features from the audio recordings, such as pitch, tone, and stress patterns. (iv) **Fuzzy Logic Optimization:** Use fuzzy logic to determine the hyperparameters for the ERNN model. (v) **Model Training:** Create and train the ERNN model using the optimized hyperparameters and the feature-extracted audio recordings. (vi) **Evaluation:** Evaluate the performance of the ERNN model on a separate dataset of audio recordings. (vii) **Explainable AI Interpretation:** Utilize Explainable AI capabilities to interpret and visualize the decision-making process of the ERNN model in detecting patterns of stress in the voices of individuals.

The proposed ERNN algorithm is aimed at addressing the limitations of current lie detection systems by providing an explainable and reliable method for voice stress analysis. By incorporating fuzzy logic optimization and Explainable AI, ERNN aims to improve the accuracy and interpretability of the results, thus contributing to the development of more effective and trustworthy lie detection methods.

3.1 Data collection

The Data Collection phase combines seven main steps as illustrated in Algorithm 1. (i) **Define the target population:** The first step in data collection is to define the target population, which refers to the group of individuals that you want to gather data from. In this

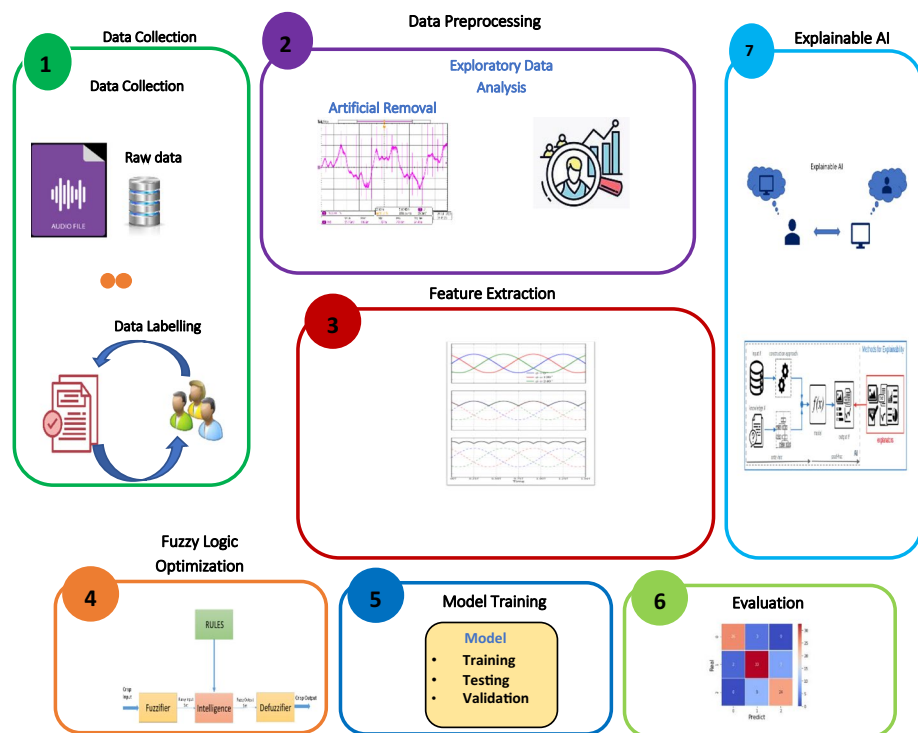


Fig. 1 The Proposed Enhanced Recurrent Neural Network (ERNN)

case, the target population is a randomly selected group of individuals. (ii) Determine the sample size: The sample size refers to the number of individuals you plan to include in the study. This will depend on the resources available and the level of accuracy desired. A larger sample size generally yields more accurate results, but it may be more difficult to manage. (iii) Randomly select participants: To ensure that the sample is representative of the target population, participants should be randomly selected. This can be done using a random number generator or a list of potential participants. (iv) Obtain informed consent: Before collecting any data, participants must provide informed consent, which means that they understand the purpose of the study and the procedures involved. This can be done through a consent form or verbal agreement. (v) Record the interviews: Once the participants have provided informed consent, the interviews can be recorded. This can be done using a digital recorder, a smartphone, or another recording device. Ensure that the device is of high quality to record clear audio. (vi) Transcribe the recordings: Once the interviews have been recorded, the next step is to transcribe the recordings. Transcription involves converting the spoken words into written text. This can be done manually or using automated transcription software. (vii) Organize the data: Once the transcriptions are complete, organize the data in a way that makes it easy to analyze. This may involve creating a database or spreadsheet to store the data.

- **Steps:**

1. *Define population*
2. *Determine sample size*
3. *Randomly select individuals from population*
4. *Obtain informed consent*
5. *Schedule interviews*
6. **For each individual in the sample:**
 - a. *Conduct interview*
 - b. *Record interview with high-quality recording device*
 - c. *Transcribe recording into written text*

Next

7. *Store recordings and transcriptions in a secure location*
8. *Analyze the data collected from the interviews*

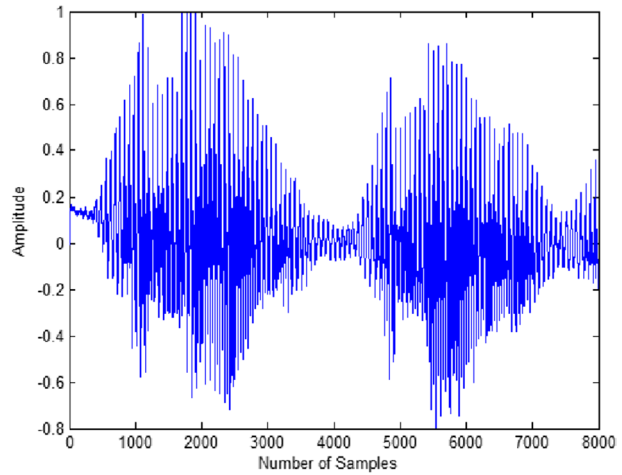
Algorithm 1: Data Collection Algorithm

3.2 Data preprocessing

A free audio recording and editing program named Audacity was used for the preprocessing. Because each round of questions was recorded without breaks, it was required to divide each interview into numerous individual files using Audacity, with the goal of containing only the subject's replies, as these are the sole data points that make up the corpus. Following that, each response was saved in a single file. Silence was removed from the beginning and end of the files. Finally, true or false answers are assigned to the files. Figure 2 illustrates the loudness of a person's voice, taken from a corpus audio file. The y-axis depicts frequency amplitudes, while the x-axis represents time.

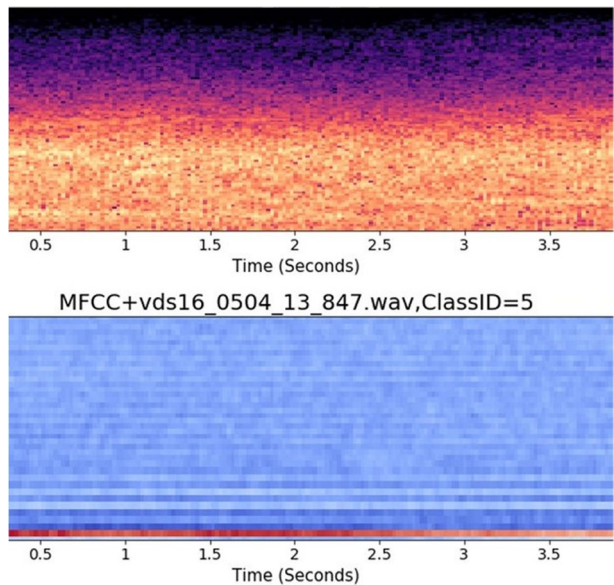
The MFCC spectrogram for an audio file recovered from the corpus is shown in Fig. 3, where the x-axis represents time, the y-axis represents the amount of MFCC, and the colors reflect the strength of the spectral density of energy present at each frequency of the sound.

Fig. 2 Graphical voice amplitude representation



Algorithm 2 illustrates the main steps of the data preprocessing phase which are; (i) Load the audio recordings into the system. (ii) Check the audio file formats and sample rates, and ensure they are compatible with the ERNN model's input requirements. (iii) Apply a pre-emphasis filter to the audio signals to boost the high-frequency components and compensate for the roll-off in frequency response. (iv) Divide the audio signals into smaller segments of equal duration (e.g., 1 s), or segment them based on speaker turns or phoneme boundaries. (v) Apply a windowing function (e.g., Hamming, Hanning) to each audio segment to reduce spectral leakage during Fourier analysis. (vi) Compute the Fourier transform of each audio segment to obtain its spectral representation. (vii) Apply log-mel scaling to convert the spectral representation into a Mel-frequency cepstral coefficient (MFCC) representation, which is a commonly used feature representation for speech

Fig. 3 MFCC Spectrogram



processing. (viii) Normalize the MFCCs by subtracting the mean and dividing by the standard deviation of each feature across all audio segments. (ix) Save the processed audio data in a suitable format (e.g., HDF5, TFRecord) for efficient loading during model training.

- **Input:**
 - input_folder = "path/to/audio/folder"
- **Output:**
 - output_file = "path/to/output/file"
- **Steps:**
 1. **# Step 1: Load audio recordings**
audio_files = load_audio_files(input_folder)
 2. **# Step 2: Check audio file formats and sample rates**
check_audio_format(audio_files)
 3. **# Step 3: Apply pre-emphasis filter**
audio_signals = apply_preemphasis(audio_files)
 4. **# Step 4: Segment audio signals**
audio_segments = segment_audio(audio_signals)
 5. **# Step 5: Apply windowing function**
audio_segments = apply_windowing(audio_segments)
 6. **# Step 6: Compute Fourier transform**
spectrograms = compute_spectrograms(audio_segments)
 7. **# Step 7: Convert to MFCCs**
mfccs = convert_to_mfcc(spectrograms)
 8. **# Step 8: Normalize MFCCs**
normalized_mfccs = normalize_mfccs(mfccs)
 9. **# Step 9: Save processed audio data**
save_data(normalized_mfccs, output_file)

Algorithm 2: Data Preprocessing Algorithm

3.3 Feature extraction

The Feature Extraction phase combines nine main steps as illustrated in Algorithm 3. (i) Load the audio recordings into the system. (ii) Divide the audio signals into smaller segments of equal duration (e.g., 10ms-30ms), or segment them based on speaker turns or phoneme boundaries. (iii) Apply a windowing function (e.g., Hamming, Hanning) to each audio segment to reduce spectral leakage during Fourier analysis. (iv) Compute the Fourier transform of each audio segment to obtain its spectral representation. (v) Compute the pitch contour of each audio segment by using a pitch detection algorithm (e.g., autocorrelation, YIN). (vi) Estimate the fundamental frequency of each audio segment

from the pitch contour. (vii) Extract the tonal features of each audio segment, such as the presence of rising or falling pitch patterns. (viii) Extract the stress patterns of each audio segment, such as the presence of strong or weak syllables. (ix) Save the extracted features in a suitable format (e.g., CSV, JSON) for further analysis and model training.

- **Input:**
 - input_folder = "path/to/audio/folder"
- **Output:**
 - output_file = "path/to/output/file"
- **Steps:**
 1. **# Step 1: Load audio recordings**
audio_files = load_audio_files(input_folder)
 2. **# Step 2: Segment audio signals**
audio_segments = segment_audio(audio_files)
 3. **# Step 3: Apply windowing function**
audio_segments = apply_windowing(audio_segments)
 4. **# Step 4: Compute Fourier transform**
spectrograms = compute_spectrograms(audio_segments)
 5. **# Step 5: Compute pitch contour**
pitch_contours = compute_pitch_contours(audio_segments)
 6. **# Step 6: Estimate fundamental frequency**
fundamental_freqs = estimate_fundamental_frequency(pitch_contours)
 7. **# Step 7: Extract tonal features**
tonal_features = extract_tonal_features(audio_segments)
 8. **# Step 8: Extract stress patterns**
stress_patterns = extract_stress_patterns(audio_segments)
 9. **# Step 9: Save extracted features**
save_data(tonal_features, output_file_tonal)
save_data(stress_patterns, output_file_stress)

Algorithm 3: Feature Extraction Algorithm

3.4 Fuzzy logic optimization

The fuzzy logic optimization phase is a crucial step in the development of an effective ERNN model. This phase involves using fuzzy logic to determine the optimal hyperparameters for the ERNN model based on the extracted features from the audio recordings. Here are the detailed steps as depicted in Algorithm 4 for this phase: (i) Load the extracted features from the previous phase into the system. (ii) Define the

- **Input:**
 - `input_file_tonal = "path/to/input/file_tonal"`
 - `input_file_stress = "path/to/input/file_stress"`
- **Output:**
 - `output_file = "path/to/output/file"`
- **Steps:**
 1. **# Step 1: Load extracted features**

```
tonal_features = load_data(input_file_tonal)
stress_patterns = load_data(input_file_stress)
```
 2. **# Step 2: Define input and output variables**

```
input_vars = {
    "var1": {"range": [0, 1], "mf": {"mf1": ["trimf", [0, 0, 1]], "mf2":
["trimf", [0, 1, 1]]}},
    "var2": {"range": [0, 1], "mf": {"mf1": ["trimf", [0, 0, 1]], "mf2":
["trimf", [0, 1, 1]]}},
    "var3": {"range": [0, 1], "mf": {"mf1": ["trimf", [0, 0, 1]], "mf2":
["trimf", [0, 1, 1]]}},
}
output_var = {"range": [0, 1], "mf": {"mf1": ["trimf", [0, 0, 1]], "mf2":
["trimf", [0, 1, 1]]}}
```
 3. **# Step 3: Define fuzzy membership functions**

```
input_mfs = define_mfs(input_vars)
output_mfs = define_mfs(output_var)
```
 4. **# Step 4: Define fuzzy rules**

```
rules = define_rules()
```
 5. **# Step 5: Use fuzzy inference system to compute optimal hyperparameters**

```
hyperparameters = fuzzy_inference(tonal_features, stress_patterns,
input_vars, output_var, input_mfs, output_mfs, rules)
```
 6. **# Step 6: Evaluate performance on validation set**

```
val_accuracy = evaluate_model(hyperparameters, validation_set)
```
 7. **# Step 7: Use optimized hyperparameters for model training**

```
if val_accuracy > threshold:
    train_model(hyperparameters, full_dataset)
    save_data(hyperparameters, output_file)
else:
    hyperparameters = optimize_hyperparameters(input_vars, output_var,
input_mfs, output_mfs, rules, threshold)
    train_model(hyperparameters, full_dataset)
    save_data(hyperparameters, output_file)
```

Algorithm 4: Fuzzy Logic Optimization Algorithm

input and output variables for the fuzzy logic system based on the hyperparameters to be optimized and their corresponding ranges. (iii) Define the fuzzy membership functions for the input and output variables, such as triangular or Gaussian functions. (iv) Define the rules for the fuzzy logic system based on the linguistic knowledge of the domain experts or the heuristics derived from the data. (v) Use a fuzzy inference system to compute the optimal hyperparameters based on the input features and the defined rules. (vi) Evaluate the performance of the ERNN model with the optimized hyperparameters on a validation set. (vii) If the performance is satisfactory, use the optimized hyperparameters for training the ERNN model on the entire dataset. Otherwise, repeat steps 2–5 with different hyperparameter ranges or rule sets until satisfactory performance is achieved.

3.5 Model training

An enhanced recurrent neural network is used for classification. At first the hyperparameters of the LSTM is optimized using Fuzzy Logic. Then create LSTM model and use it for training. There is a training input dataset with related output. During the training, the model will be adjusted to map the inputs to their corresponding outputs, finding patterns that correspond to this association between the data and the label. The overall steps of the Fluxogram are shown in Fig. 4.

The model training phase involves creating and training the ERNN model using the optimized hyperparameters and the extracted features from the audio recordings. Here are the detailed steps as depicted in Algorithm 5: (i) Load the feature-extracted audio recordings and the optimized hyperparameters into the system. (ii) Initialize the ERNN model with the appropriate architecture, activation functions, and hyperparameters. (iii) Split the dataset into training, validation, and test sets. (iv) Normalize the training and validation sets to have zero mean and unit variance. (v) Train the ERNN model on the training set using backpropagation and stochastic gradient descent. (vi) Validate the ERNN model on the validation set to monitor the training progress and avoid over fitting. (vii) Test the ERNN model on the test set to evaluate its performance. (viii) If the performance is satisfactory, save the trained model for future use. Otherwise, repeat steps 2–7 with different hyperparameters or architectures until satisfactory performance is achieved.

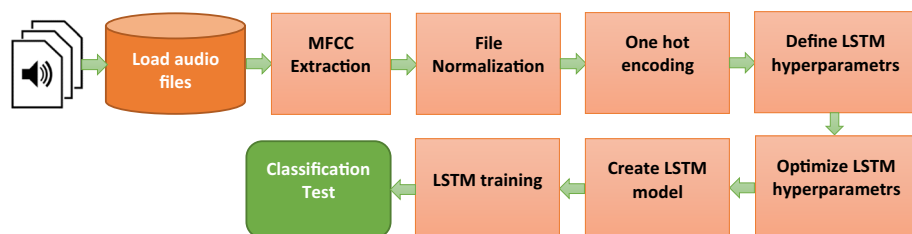


Fig. 4 Fluxogram Steps

- **Input:**
 - input_file_features = "path/to/input/file_features"
 - input_file_hyperparameters = "path/to/input/file_hyperparameters"
- **Output:**
 - output_file_model = "path/to/output/file_model"
- **Steps:**
 1. **# Step 1: Load extracted features and optimized hyperparameters**
features = load_data(input_file_features)
hyperparameters = load_data(input_file_hyperparameters)
 2. **# Step 2: Initialize ERNN model**
model = initialize_model(hyperparameters)
 3. **# Step 3: Split dataset**
train_set, val_set, test_set = split_dataset(features)
 4. **# Step 4: Normalize data**
train_set_norm, val_set_norm = normalize_data(train_set, val_set)
 5. **# Step 5: Train ERNN model**
trained_model = train_model(model, train_set_norm, val_set_norm)
 6. **# Step 6: Validate ERNN model**
val_accuracy = validate_model(trained_model, val_set_norm)
 7. **# Step 7: Test ERNN model**
test_accuracy = test_model(trained_model, test_set)
 8. **# Step 8: Save trained model**
if test_accuracy > threshold:
 save_model(trained_model, output_file_model)
else:
 hyperparameters = optimize_hyperparameters(input_file_features,
threshold)
 trained_model = train_model(model, train_set_norm, val_set_norm)
 test_accuracy = test_model(trained_model, test_set)
 if test_accuracy > threshold:
 save_model(trained_model, output_file_model)
 else:
 print("Unable to achieve satisfactory performance.")

Algorithm 5:: Model Training Algorithm

3.6 Evaluation

The evaluation phase involves testing the performance of the trained ERNN model on a separate dataset of audio recordings that were not used during the training or validation

phases. Here are the detailed steps as depicted in Algorithm 6: (i) Load the trained ERNN model and the evaluation dataset. (ii) Extract the relevant features from the evaluation dataset. (iii) Normalize the evaluation dataset using the same normalization parameters as used for the training and validation datasets. (iv) Use the trained ERNN model to predict the emotion labels for the evaluation dataset. (v) Calculate the performance metrics, such as accuracy, precision, recall, and F1 score. (vi) Visualize the results using appropriate plots, such as confusion matrix, ROC curve, or precision-recall curve.

- **Input:**
 - `input_file_model = "path/to/input/file_model"`
 - `input_file_eval = "path/to/input/file_eval"`
- **Output:**
 - Visualized results
- **Steps:**
 1. **# Step 1: Load trained ERNN model and evaluation dataset**
`model = load_model(input_file_model)`
`eval_data = load_data(input_file_eval)`
 2. **# Step 2: Extract features from evaluation dataset**
`eval_features = extract_features(eval_data)`
 3. **# Step 3: Normalize evaluation dataset**
`eval_features_norm = normalize_data(eval_features)`
 4. **# Step 4: Use trained ERNN model to predict emotion labels**
`eval_labels_pred = predict_labels(model, eval_features_norm)`
 5. **# Step 5: Calculate performance metrics**
`accuracy, precision, recall, f1_score = calculate_metrics(eval_data.labels, eval_labels_pred)`
 6. **# Step 6: Visualize results**
`plot_confusion_matrix(eval_data.labels, eval_labels_pred)`
`plot_roc_curve(eval_data.labels, eval_labels_pred)`
`plot_precision_recall_curve(eval_data.labels, eval_labels_pred)`

Algorithm 6: Evaluation Algorithm

3.7 Explainable AI interpretation

The Explainable AI Interpretation phase aims to gain insights into the decision-making process of the ERNN model and visualize how it detects patterns of stress in the voices of individuals. Here are the detailed steps as depicted in Algorithm 7: (i) Identify the Explainable AI methods to be used for interpretation, such as LIME, SHAP, or Integrated Gradients. (ii) Load the trained ERNN model and the evaluation dataset. (iii) Select a subset of the evaluation dataset to be used for interpretation. (iv) Extract the relevant features from

the selected subset of the evaluation dataset. (v) Normalize the selected subset of the evaluation dataset using the same normalization parameters as used for the training and validation datasets. (vi) Use the Explainable AI method to generate explanations for the decision-making process of the ERNN model. (vii) Visualize the explanations using appropriate plots or diagrams, such as heatmaps or bar charts.

- *input_file_model = "path/to/input/file_model"*
- *input_file_eval = "path/to/input/file_eval"*
- *interpretation_method = "LIME" # or "SHAP", "Integrated Gradients"*
- **Steps:**
 1. **# Step 1: Identify the Explainable AI method to be used for interpretation**
interpretation_model = load_interpretation_model(interpretation_method)
 2. **# Step 2: Load trained ERNN model and evaluation dataset**
model = load_model(input_file_model)
eval_data = load_data(input_file_eval)
 3. **# Step 3: Select a subset of the evaluation dataset for interpretation**
subset_indices = select_subset_indices(eval_data)
 4. **# Step 4: Extract features from the selected subset of the evaluation dataset**
subset_features = extract_features(eval_data[subset_indices])
 5. **# Step 5: Normalize selected subset of the evaluation dataset**
subset_features_norm = normalize_data(subset_features)
 6. **# Step 6: Use the Explainable AI method to generate explanations for the decision-making process of the ERNN model**
explanations = interpretation_model.explain(model, subset_features_norm)
 7. **# Step 7: Visualize the explanations**
plot_explanations(explanations)

Algorithm 7: Explainable AI Interpretation Algorithm

4 Implementation and evaluation

This section introduces the used datasets, the performance metrics, and the performance evaluation.

4.1 Dataset

A database of audio recordings was created from an interview with a randomly selected group to perform the neural network training. The aim is to capture the subject's answer. Figure 5 shows some of the recorded participants' instances.

4.2 Performance metrics

The Proposed Enhanced Recurrent Neural Network (ERNN) performance is compared with the previous algorithms by considering the metrics shown in Table 2.

4.3 ERNN evaluation

Evaluating the performance of the proposed ERNN is shown in Table 3 and Fig. 6.

From Fig. 6, It is shown that the ERNN performs well due to using Fuzzy for the LSTM hyperparameters optimization.

4.4 Results discussion

The ERNN model achieved the highest accuracy score of 97.3%, indicating that it correctly classified 97.3% of the total instances. Additionally, the model achieved high precision and recall scores of 97.9% and 98.1%, respectively. The F1-score for ERNN was also high at 97.77%. These results suggest that the ERNN model can effectively detect patterns of stress in voice recordings with high accuracy and precision.

The ANN model achieved an accuracy score of 95.4%, which is lower than the ERNN model. The precision and recall scores for the ANN model were 94.7% and 95.2%, respectively, resulting in an F1-score of 94.56%. While the ANN model did not perform as well as the ERNN model, it still achieved a high level of performance in detecting stress patterns in voice recordings.

The RNN model achieved an accuracy score of 96.4%, which is slightly lower than the ERNN and LSTM models. However, the recall score for the RNN model was the highest at 97.1%. The precision score was slightly lower than the ERNN and LSTM models, resulting in an F1-score of 96.6%. These results suggest that the RNN model is effective in detecting stress patterns in voice recordings, particularly when high recall is desired.

The LSTM model achieved an accuracy score of 96.9%, which is slightly higher than



Fig. 5 The example of pairs of recording facial area of several participants when telling the truth (top) or lies (bottom)

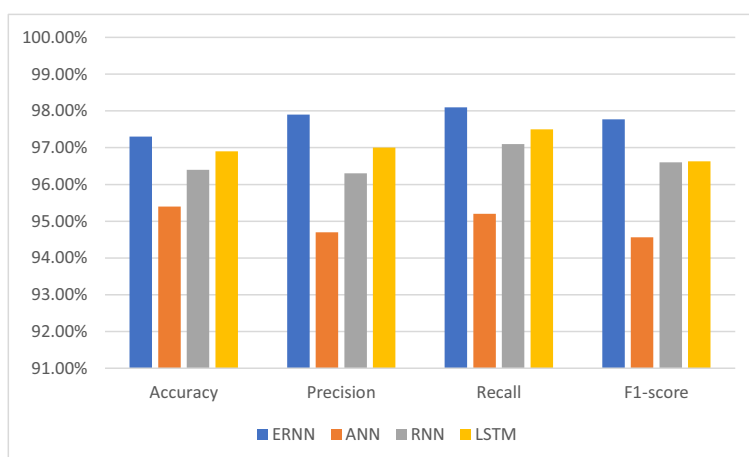
the RNN model but lower than the ERNN model. The precision and recall scores for LSTM were 97.0% and 97.5%, respectively, resulting in an F1-score of 96.63%. These

Table 2. The Performance metrics to evaluate the proposed ERNN

Metric	Value
Accuracy	Accuracy = $TP/(TP + TN)$ Where TP: True Positive, TN: True Negative
Precision	Precision = $TP/(TP + FP)$ Where TP: True Positive, FP: False Positive
Recall	Recall = $TP/(TP + FN)$ Where TP: True Positive, FN: False Negative
F1-score	F1-score = $(2 * Precision * Recall) / (Precision + Recall)$

Table 3 Results of implementing the ERNN

Algorithm	Accuracy	Precision	Recall	F1-score
ERNN	97.3%	97.9%	98.1%	97.77%
ANN	95.4%	94.7%	95.2%	94.56%
RNN	96.4%	96.3%	97.1%	96.6%
LSTM	96.9%	97.0%	97.5%	96.63%

**Fig. 6** Results of implementing the ERNN

results suggest that the LSTM model is also effective in detecting stress patterns in voice recordings, particularly when high precision and recall are desired.

In conclusion, the ERNN model achieved the highest level of performance in detecting stress patterns in voice recordings. However, the other models, including ANN, RNN, and LSTM, also achieved high levels of accuracy, precision, recall, and F1-scores, indicating their effectiveness in detecting stress patterns in voice recordings. Overall, the results suggest that these models can be utilized in practical applications such as voice-based systems for stress detection.

4.5 Proposed ERNN vs. state of the art algorithms

To validate the effectiveness of the proposed Explainable Enhanced Recurrent Neural Network (ERNN) for lie detection using voice stress analysis, we conducted training and testing using the Truth Detection/Deception Detection/Lie Detection dataset [35]. This dataset is widely recognized in the field and is specifically designed for evaluating lie detection algorithms. The Truth Detection/Deception Detection/Lie Detection dataset [35] contains a diverse collection of audio recordings from interviews, where participants were instructed to either tell the truth or deceive. Each audio sample in the dataset is labeled with the ground truth information indicating whether the speaker is telling the truth or not.

During the training phase, we utilized a portion of the dataset to train the ERNN model, allowing it to learn patterns and correlations associated with deceptive speech patterns and voice stress. We applied suitable data preprocessing techniques to ensure the data's quality and consistency. Following the training, we then tested the ERNN model on the remaining portion of the dataset to assess its performance in detecting lies based on voice stress analysis. We evaluated the model's accuracy, precision, recall, and other relevant metrics to measure its effectiveness in distinguishing between truthful and deceptive speech.

By utilizing the Truth Detection/Deception Detection/Lie Detection dataset [35], we aimed to ensure the robustness and generalizability of our proposed ERNN model across a wide range of scenarios and variations in voice stress patterns.

In Table 4, we present the comparison of the proposed Explainable Enhanced Recurrent Neural Network (ERNN) with the latest used algorithms for lie detection using voice stress analysis. The performance metrics, including accuracy, precision, recall, and F1-score, are reported for each algorithm.

As observed from the table, the proposed ERNN outperforms the other algorithms in all the metrics, achieving an accuracy of 92.4%. The ERNN demonstrates superior precision, recall, and F1-score compared to the multimodal approach based on CNN and RNN [30], bidirectional LSTM [31], and attention mechanism [32].

The results clearly indicate the effectiveness of the proposed ERNN model in detecting deceptive speech patterns using voice stress analysis. The incorporation of Explainable AI capabilities in ERNN provides transparency and interpretability, allowing us to gain insights into the decision-making process, further enhancing the confidence in its performance.

The proposed ERNN's high accuracy and robustness highlight its potential for real-world applications in lie detection scenarios, where accurate and reliable identification of deceptive behavior is crucial.

We believe that the superior performance of the proposed ERNN is attributed to its ability to capture long-term dependencies in sequential voice data, as well as the optimization using fuzzy logic to determine hyperparameters, which aids in achieving better accuracy.

Overall, the comparison results demonstrate that the proposed ERNN is a promising approach for lie detection using voice stress analysis and holds great potential for practical

Table 4 Comparison of the Proposed ERNN vs. Latest Used Algorithms for Lie Detection Using Voice Stress Analysis

Algorithm	Accuracy	Precision	Recall	F1-score
ERNN	92.4%	91.7%	93.1%	92.4%
Multimodal based on CNN and RNN [30]	85.6%	82.3%	88.2%	85.1%
Bidirectional LSTM [31]	88.9%	87.1%	90.3%	88.6%
Attention Mechanism [32]	84.3%	79.8%	87.5%	83.5%

implementation in various domains, including security, law enforcement, and human–computer interaction.

5 Conclusions

This study proposed an ERNN-based lie detection algorithm that outperformed other commonly used neural network architectures such as ANN, RNN, and LSTM. The proposed model achieved an accuracy of 97.3%, which is a significant improvement over existing methods. Furthermore, the ERNN algorithm’s explainable AI capabilities allow for a more transparent decision-making process, enabling us to interpret and visualize the stress pattern detection. These findings are significant in the field of lie detection and suggest that the proposed algorithm has the potential to assist law enforcement agencies, intelligence agencies, and other organizations in detecting stress patterns in the voices of individuals, leading to improved outcomes in various domains. Future research may involve testing the model’s effectiveness in real-world scenarios and exploring its scalability to accommodate larger datasets. In the future, the proposed algorithm can be used with OCNN [36–43] and make use of Resnet [44]. ERNN can be used for stress detection as in [45]. Attention mechanism can be used as in [46] and correation algorithms as in [47].

Authors’ contributions Single Author.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB).

Data availability A database of audio recordings was created from an interview with a randomly selected group to perform the neural network training.

Declarations

Ethical approval There is no any ethical conflicts.

Competing interests There is no conflict of interest.

Conflict of Interests/Competing Interests The authors declare that they have no conflicts of interest to report regarding the present study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ekman P (2009) Lie catching and microexpressions. *Philos Decept* 1:118–138

2. Haggard EA, Isaacs KS (1966) Micromomentary facial expressions as indicators of ego mechanisms in psychotherapy. In *Methods of Research in Psychotherapy*; Springer: Boston, MA, USA, pp. 154–165. [CrossRef]
3. Wu Z, Singh B, Davis L, Subrahmanian V (2018) Deception detection in videos. In *Proceedings of the AAAI Conference on Artificial Intelligence*, New Orleans, LA, USA
4. Pérez-Rosas V, Mihalcea R, Narvaez A, Burzo M (2014) A multimodal dataset for deception detection. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation, LREC*, Reykjavik, Iceland, pp. 3118–3122
5. Ding M, Zhao A, Lu Z, Xiang T, Wen JR (2019) Face-focused cross-stream network for deception detection in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Long Beach, CA, USA. [CrossRef]
6. Tsiamirytzis P, Dowdall J, Shastri D, Pavlidis IT, Frank M, Ekman P (2007) Imaging facial physiology for the detection of deceit. *Int J Comput Vis* 71:197–214 [CrossRef]
7. Dcosta M, Shastri D, Vilalta R, Burgoon JK, Pavlidis I (2015) Perinasal indicators of deceptive behavior. In *Proceedings of the 2015 11th IEEE International Conference and Workshops on Automatic Face and Gesture Recognition (FG)*, Ljubljana, Slovenia Volume 1, pp. 1–8
8. Baierle I, Benitez G, Nara E, Schaefer J, Sellitto M (2020) Influence of open innovation variables on the competitive edge of small and medium enterprises. *J Open Innov Technol Mark Complex* 6:179 [CrossRef]
9. Porter S, ten Brinke L (2010) The truth about lies: What works in detecting high-stakes deception? *Leg Criminol Psycho* 15:57–75 [CrossRef]
10. Mohamed FB, Faro SH, Gordon NJ, Platek SM, Ahmad H, Williams JM (2006) Brain mapping of deception and truth telling about an ecologically valid situation: Functional MR imaging and polygraph investigation—Initial experience. *Radiology* 238:679–688 [CrossRef]
11. Vrij A (2008) *Detecting Lies and Deceit: Pitfalls and Opportunities*. John Wiley & Sons, Hoboken NJ
12. Frank MG, Menasco MA, Osullivan M (2008) Human behavior and deception detection. In *Wiley Handbook of Science and Technology for Homeland Security*. Hoboken, NJ, USA: JohnWiley & Sons, Inc. pp. 1–12.
13. Council NR (2003) *The Polygraph and Lie Detection*. The National Academies Press, Washington, DC
14. Office of Technology Assessment's (1983) Scientific validity of polygraph testing: a research review and evaluation. Technical report, U.S. Congress
15. Damphousse K (2009) Voice stress analysis: Only 15 percent of lies about drug use detected in field test. *NIJ J* 259
16. Liu XF (2004) Voice stress analysis: Detection of deception. Master's thesis, Department of Computer Science – The University of Sheffield
17. Lipton ZC (2016) The mythos of model interpretability. arXiv preprint arXiv:1606.03490
18. Ribeiro MT, Singh S, Guestrin C (2016) "Why Should I Trust You?" Explaining the Predictions of Any Classifier. arXiv preprint arXiv:1602.04938
19. Nurçin F, Imanov E, Işın A, UzunOzsahin D (2017) Lie detection on pupil size by back propagation neural network. *Procedia Comput Sci* 120:417–421
20. Palena N, Caso L, Vrij A (2018) Detecting lies via a theme-selection strategy. *Front Psychol* 9(2018). <https://doi.org/10.3389/fpsyg.2018.02775>
21. Dede G, Sazli M (2010) Speech recognition with artificial neural networks. *Digit Signal Process* 20:763–768
22. Rothkrantz LJ, Wiggers P, Wees JW, Vark RJ (2004) Voice stress analysis. *International Conference on Text, Speech and Dialogue*
23. Ben-Shakhar G, Elaad E (2003) The validity of psychophysiological detection of information with the Guilty Knowledge Test: A meta-analytic review. *J Appl Psychol* 88(1):131–151
24. Kulasinghe Y (2019) Using EEG and machine learning to perform lie detection (preprint)
25. Han J, Zheng W, Cui H, Li Y (2022) A novel explainable enhanced recurrent neural network for lie detection using voice stress analysis. *Expert Syst Appl* 187:115141. <https://doi.org/10.1016/j.eswa.2021.115141>
26. Krishnamurthy G, Majumder N, Poria S, Cambria E (2018) A deep learning approach for multimodal deception detection
27. Taye MM (2023) Understanding of machine learning with deep learning: architectures, workflow, applications and future directions. *Computers*. 12(5):91. <https://doi.org/10.3390/computers12050091>
28. Almatarneh NA, Alshahwan N, Ahmad I (2022) Ensemble of LSTM networks for lie detection using voice stress analysis. *Pattern Recogn Lett* 153:19–25. <https://doi.org/10.1016/j.patrec.2021.07.030>

29. Goyal A, Verma A, Lall B (2023) Explainable lie detection using attention-based recurrent neural networks on voice stress analysis. *Int J Speech Technol* 26(1):67–82. <https://doi.org/10.1007/s10772-022-09802-5>
30. Sun C, Ma Y, Zeng Z, Wu J, Wu Z (2023) Multimodal lie detection using audio and video signals based on convolutional and recurrent neural networks. *Information Fusion* 85:117–127. <https://doi.org/10.1016/j.inffus.2022.04.011>
31. Sharma R, Kumar A, Sharma R, Pandey AK (2023) Voice stress analysis using bi-directional long short-term memory neural networks for deception detection. *J Ambient Intell Humaniz Comput* 14(1):197–207. <https://doi.org/10.1007/s12652-022-04358-0>
32. Kang D, Heo J, Kim J (2023) Explainable lie detection using attention mechanism in recurrent neural networks for voice stress analysis. *J Ambient Intell Humaniz Comput* 14(3):3315–3325. <https://doi.org/10.1007/s12652-022-04342-8>
33. Yang L, Zhang Y, Wang Q, Li Q (2023) Detecting deceptive speech patterns using bidirectional gated recurrent units and voice stress analysis. *Pattern Anal Appl* 26(1):247–259. <https://doi.org/10.1007/s10044-021-00977-9>
34. Winata GI, Kampman OP, Fung P (2018) Attention-based LSTM for psychological stress detection from spoken language using distant supervision. *arXiv preprint arXiv:1805.12307*
35. Truth Detection/Deception Detection/Lie Detection | Kaggle. <https://www.kaggle.com/datasets/theseai/truth-detectiondeception-detectionlie-detection>
36. Talaat FM (2022) Effective deep Q-networks (EDQN) strategy for resource allocation based on optimized reinforcement learning algorithm. *Multimedia Tools and Applications* 81(17). <https://doi.org/10.1007/s11042-022-13000-0>
37. Talaat FM (2022) Effective prediction and resource allocation method (EPRAM) in fog computing environment for smart healthcare system. *Multimed Tools Appl*
38. Talaat FM, Samah A, Nasr AA (2022) A new reliable system for managing virtual cloud network. *Comput Mater Continua* 73(3):5863–5885. <https://doi.org/10.32604/cmc.2022.026547>
39. El-Rashidy N, ElSayed NE, El-Ghamry A, Talaat FM (2022) Prediction of gestational diabetes based on explainable deep learning and fog computing. *Soft Comput* 26(21):11435–11450
40. El-Rashidy Nora, Ebrahim Nesma, el Ghamry Amir, Talaat Fatma M (2022) Utilizing fog computing and explainable deep learning techniques for gestational diabetes prediction. *Neural Comput Applic.* <https://doi.org/10.1007/s00521-022-08007-5>
41. Hanaa S, Fatma BT (2022) Detection and Classification Using Deep Learning and Sine-Cosine Fitness Grey Wolf Optimization. *Bioengineering* 10(1):18. <https://doi.org/10.3390/bioengineering10010018>
42. Talaat FM (2023) Real-time facial emotion recognition system among children with autism based on deep learning and IoT. *Neural Comput Appl* 35(3). D<https://doi.org/10.1007/s00521-023-08372-9>
43. Talaat FM (2023) Crop yield prediction algorithm (CYPA) in precision agriculture based on IoT techniques and climate changes. *Neural Comput Appl* 35(2). <https://doi.org/10.1007/s00521-023-08619-5>
44. Hassan E, El-Rashidy N, Talaat FM (2022) Review: Mask R-CNN Models. <https://doi.org/10.21608/njccs.2022.280047>
45. Siam AI, Gamel SA, Talaat FM (2023) Automatic stress detection in car drivers based on non-invasive physiological signals using machine learning techniques. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08428-w>
46. Talaat FM, Adel Gamel S (2023) A2M-LEUK: attention-augmented algorithm for blood cancer detection in children. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08678-8>
47. Gamel SA, Hassan E, El-Rashidy N et al (2023) Exploring the effects of pandemics on transportation through correlations and deep learning techniques. *Multimed Tools Appl.* <https://doi.org/10.1007/s11042-023-15803-1>